

- **Name:** Aj Kean Edoria
- **Term:** Term 3 2025-2026

Machine Problem 003: Transfer Function

The [Python Control Systems Library](#) provides basic tools for the analysis and design of linear feedback control systems. The library provides tools to specify transfer function and state space models, manipulate models using block diagram algebra, stability analysis, and perform time and frequency domain simulation.

The purpose of these notes is to provide a quick start with the Python Control Systems Library. Consult the [Python Control Systems Documentation](#) for more details.

Installation

The [Python Control Systems Library](#) is not, unfortunately, a standard part of most standard Python distributions. On most systems, the following commands will perform the required one-time installation of the necessary software.

To install both the Control Systems library and Slycot in an existing conda environment, run:

```
!conda install -c conda-forge control slycot
```

```
In [5]: !conda install -c conda-forge slycot
!pip install control
!pip install seaborn
```

^C

Requirement already satisfied: control in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (0.10.2)

Requirement already satisfied: numpy>=1.23 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from control) (2.4.6)

Requirement already satisfied: scipy>=1.8 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from control) (1.17.1)

Requirement already satisfied: matplotlib>=3.6 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from control) (3.10.9)

Requirement already satisfied: contourpy>=1.0.1 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (1.3.3)

Requirement already satisfied: cyclor>=0.10 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (1.5.0)

Requirement already satisfied: packaging>=20.0 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (26.0)

Requirement already satisfied: pillow>=8 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (12.2.0)

Requirement already satisfied: pyparsing>=3 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.6->control) (1.17.0)

Requirement already satisfied: seaborn in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (0.13.2)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from seaborn) (2.4.6)

Requirement already satisfied: pandas>=1.2 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from seaborn) (3.0.3)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from seaborn) (3.10.9)

Requirement already satisfied: contourpy>=1.0.1 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)

Requirement already satisfied: cyclor>=0.10 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)

Requirement already satisfied: packaging>=20.0 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.0)

Requirement already satisfied: pillow>=8 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.2.0)

Requirement already satisfied: pyparsing>=3 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: tzdata in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from pandas>=1.2->seaborn) (2026.2)

Requirement already satisfied: six>=1.5 in C:\ProgramData\miniconda3\envs\control-system-cpe2a\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

Library Usage

The control systems library is designed to work with a simplified syntax where libraries are imported without the standard prefixes.

```
In [3]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import control.matlab as control
```

Demonstration

Step Response of Transfer Functions

The following cells demonstrate the use of two functions in the control systems library, `tf` and `step`. Suppose a signal $y(s)$ is related to an input $u(s)$ by the formula

$$y(s) = \underbrace{\frac{4.3}{3.2s + 1}}_{G(s)} u(s)$$

The transfer function

$$G(s) = \frac{4.3}{3.2s + 1}$$

is represented in the control system library using `tf(num,den)` where `num` and `den` list the coefficients of the numerator and denominator polynomials.

```
In [4]: G = control.tf([4.3],[3.2, 1])
print(G)
```

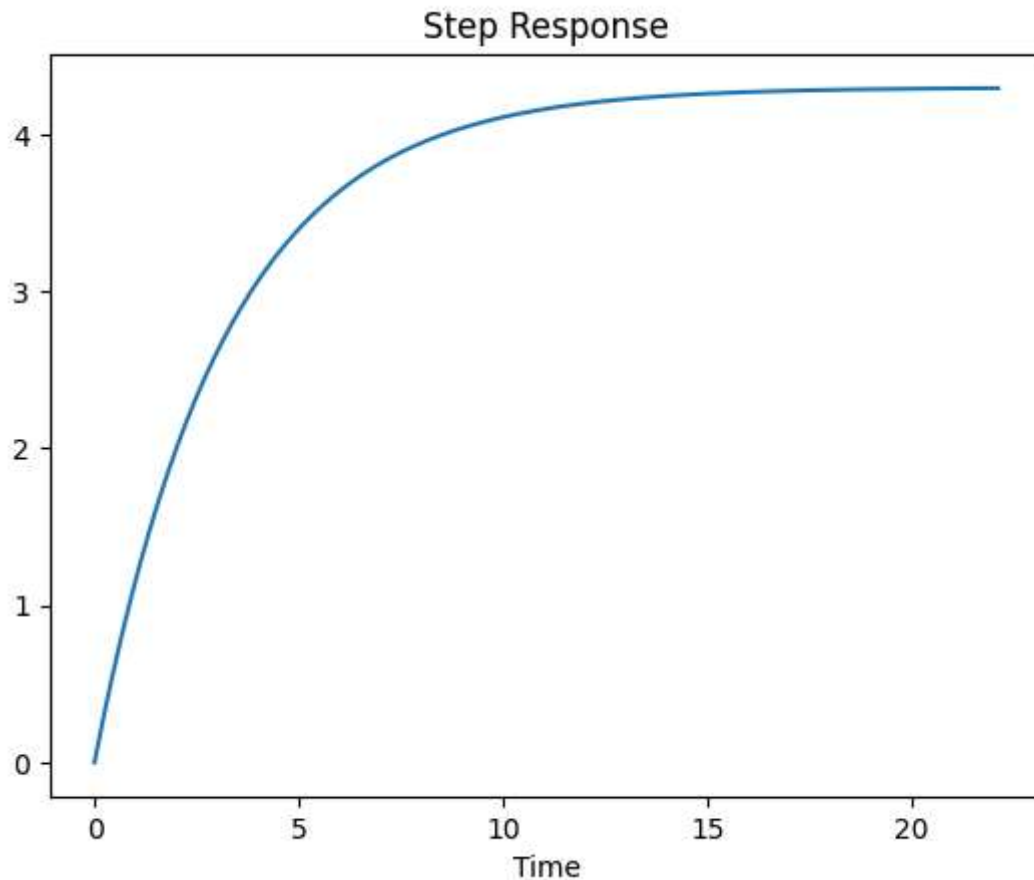
```
<TransferFunction>: sys[0]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
```

```
      4.3
-----
    3.2 s + 1
```

The step response is created and plotted as

```
In [6]: y,t = control.step(G)
plt.plot(t,y)
plt.xlabel('Time')
plt.title('Step Response')
```

Out[6]: Text(0.5, 1.0, 'Step Response')



Transfer Functions in Series

Models for complex systems are generally constructed by combining models of simpler elements. Consider, for example, a serial connection of two transfer functions

$$y_1(s) = G_1(s)u(s)$$

$$y_2(s) = G_2(s)y_1(s)$$

which can be diagrammed as

$$u(s) \longrightarrow \boxed{G_1(s)}$$

The serial composition of two transfer functions can be written

$$y_2(s) = \underbrace{G_2(s)G_1(s)}_{G(s)}u(s)$$

where the product $G(s) = G_2(s)G_1(s)$ represents the transfer function of the combined system. In terms of a block diagram

$$u(s) \longrightarrow \boxed{G(s)}$$

The product $G(s) = G_2(s)G_1(s)$

is computed by taking the product of the numerator and denominator polynomials, respect

$$G_1(s) = \frac{12.3}{10s+1} \text{ and } G_2(s) = \frac{4}{15s+1}$$

Then

$$\begin{aligned} G(s) &= G_2(s)G_1(s) \\ &= \frac{4}{15s+1} \times \frac{12.3}{10s+1} = \frac{4 \times 12.3}{(15s+1)(10s+1)} = \frac{49.2}{150s^2 + 25s + 1} \end{aligned}$$

We can verify this calculation using the control systems library.

```
In [7]: G1 = control.tf([12.3],[10,1])
        G2 = control.tf([4],[15,1])

        G = G2 * G1

        print(G)

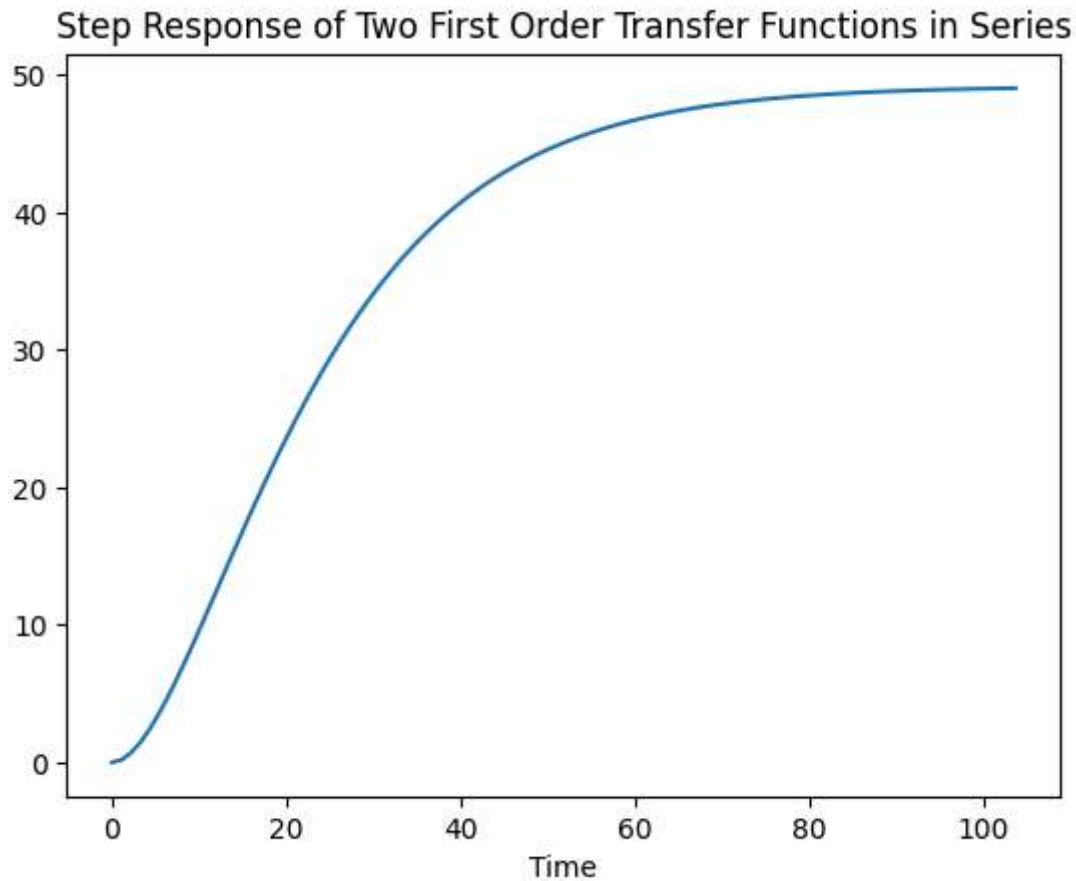
        y,t = control.step(G)
        plt.plot(t,y)
        plt.xlabel('Time')
        plt.title('Step Response of Two First Order Transfer Functions in Series')
```

```
<TransferFunction>: sys[5]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
```

```

          49.2
-----
150 s^2 + 25 s + 1
```

```
Out[7]: Text(0.5, 1.0, 'Step Response of Two First Order Transfer Functions in Series')
```



Second Order Transfer Functions

The control systems library is particularly helpful when working with second and higher order transfer functions. For

$$G(s) = \frac{3}{4s^2 + s + 1}$$

```
In [8]: R = control.tf([3.],[4.,1.,1.])
print(R)

y,t = control.step(R)
plt.plot(t,y)
plt.xlabel('Time')
plt.title('Response of a Second Order System')
```

```
<TransferFunction>: sys[8]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']
```

$$\frac{3}{4s^2 + s + 1}$$

```
Out[8]: Text(0.5, 1.0, 'Response of a Second Order System')
```

